

Fundação Escola de Comércio Álvares Penteado

Douglas Macario

Eric Carvalho

Gustavo Cocoloti

Vinicius Borges

Software de Gestão de Jogos

11/05/2026

Software de Gestão de Jogos

Grupo: Cyber Builders

Professor orientador: Eduardo Savino

Aplicação da Estrutura de Modularização no Projeto

Durante o desenvolvimento do projeto, utilizamos a modularização para deixar o sistema mais organizado e facilitar tanto a programação quanto a manutenção do código. A modularização consiste em dividir o programa em partes menores, onde cada parte possui uma função específica dentro da aplicação.

No projeto, começamos utilizando o “namespace MeuPI”, responsável por organizar as classes do sistema em um mesmo grupo. Isso ajudou a manter o projeto mais estruturado e evitou conflitos entre nomes de classes.

Também utilizamos a estrutura “partial class Form1”, que permitiu separar a interface gráfica da lógica do sistema. Dessa forma, a parte visual ficou em um arquivo e a parte funcional em outro, deixando o código mais limpo, organizado e fácil de entender.

A interface gráfica ficou responsável pelos componentes visuais do sistema, como:

- botões;
- labels;
- caixas de texto.

Já a parte lógica ficou responsável pelas funcionalidades do programa, como conexão com o banco de dados, validação das informações e inserção dos dados.

Outro ponto importante da modularização foi a criação de procedimentos específicos para cada função do sistema. Em vez de colocar tudo em um único bloco de código, cada tarefa foi separada em um procedimento diferente.

O procedimento `ConectarBanco()` foi criado para realizar a conexão com o banco de dados do SQLite. Dentro dele também foram utilizados outros procedimentos auxiliares, como `AtivarChavesEstrangeiras()` e `CriarTabelas()`, ajudando na organização do código.

```
private bool ConectarBanco()
{
    try
    {
        string caminhoBanco = txtArquivoDB.Text.Trim();

        if (string.IsNullOrEmpty(caminhoBanco))
            caminhoBanco = "Messier.db";

        string strConexao = $"Data Source={caminhoBanco}";

        conexao = new SQLiteConnection(strConexao);
        conexao.Open();

        AtivarChavesEstrangeiras();
        CriarTabelas();

        MessageBox.Show("Conexão concluída!");
        return true;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Erro ao conectar: " + ex.Message);
        return false;
    }
}
```

O procedimento `AtivarChavesEstrangeiras()` ficou responsável por ativar as restrições de chave estrangeira do banco de dados, enquanto o procedimento `CriarTabelas()` foi utilizado para criar automaticamente a tabela `JOGOS`.

Também foi utilizado o procedimento `ValidarCodigo()`, responsável por verificar se o código informado pelo usuário é válido antes de realizar a inserção no banco. Isso ajudou a evitar erros durante o funcionamento do sistema.

```

private bool ValidarCodigo(out int codigo)
{
    if (!int.TryParse(txtCodigo.Text, out codigo))
    {
        MessageBox.Show("Digite um código válido!");
        return false;
    }

    return true;
}

1 referência
private bool ValidarStatus(string status)
{
    foreach (string s in statusValidos)
    {
        if (s == status.ToUpper())
            return true;
    }

    MessageBox.Show("Digite apenas A ou I no status!");
    return false;
}

```

O procedimento `InserirJogo()` foi desenvolvido para realizar a inserção das informações no banco de dados, organizando toda a parte responsável pelo comando SQL de cadastro.

```

private void InserirJogo(int codigo, string nome,
                        string descricao,
                        string tema,
                        string status)
{
    if (conexao == null)
    {
        MessageBox.Show("Conecte primeiro!");
        return;
    }

    string sql = @"
        INSERT INTO JOGOS
        (codigo, nome, descricao, tema, status)
        VALUES
        (@codigo, @nome, @descricao, @tema, @status)";

    using (SQLiteCommand cmd = conexao.CreateCommand())
    {
        cmd.CommandText = sql;

        cmd.Parameters.AddWithValue("@codigo", codigo);
        cmd.Parameters.AddWithValue("@nome", nome);
        cmd.Parameters.AddWithValue("@descricao", descricao);
        cmd.Parameters.AddWithValue("@tema", tema);
        cmd.Parameters.AddWithValue("@status", status);

        cmd.ExecuteNonQuery();
    }

    MessageBox.Show("Jogo inserido com sucesso!");
    LimparCampos();
}

```

Outro procedimento utilizado foi o `LimparCampos()`, responsável por limpar os campos do formulário após a inserção dos dados, melhorando a reutilização do código e evitando repetição de comandos.

```
private void LimparCampos()
{
    txtCodigo.Clear();
    txtNome.Clear();
    txtDesc.Clear();
    txtTema.Clear();
    txtStatus.Clear();

    txtCodigo.Focus();
}
```

Além disso, os eventos dos botões também foram separados em procedimentos próprios, como `btnConectar_Click()` e `btnInserir_Click()`, responsáveis pelas ações executadas quando o usuário interage com os botões do sistema.

Essa divisão em procedimentos deixou o código mais organizado, facilitou a leitura e ajudou na manutenção do sistema durante o desenvolvimento do projeto.

```
private void btnConectar_Click(object sender, EventArgs e)
{
    ConectarBanco();
}
```

```
private void btnInserir_Click(object sender, EventArgs e)
{
    if (!ValidarCodigo(out int codigo))
        return;

    if (!ValidarStatus(txtStatus.Text))
        return;

    try
    {
        InserirJogo(
            codigo,
            txtNome.Text,
            txtDesc.Text,
            txtTema.Text,
            txtStatus.Text
        );
    }
}
```

Métodos e Procedimentos Utilizados no Projeto	
Método / Procedimento	Função no Sistema
ConectarBanco()	Realiza a conexão com o banco de dados SQLite. .
AtivarChavesEstrangeiras()	Ativa as restrições de chave estrangeira no banco de dados.
CriarTabelas()	Cria a tabela JOGOS no banco de dados.
ValidarCodigo()	Verifica se o código digitado pelo usuário é válido. .
InserirJogo()	Realiza a inserção das informações no banco de dados.
LimparCampos()	Realiza a inserção das informações no banco de dados.
btnConectar_Click()	Executa a conexão ao clicar no botão “Conectar”.
btnInserir_Click()	Executa a inserção dos dados ao clicar no botão “Inserir”.
txtNome_Click()	Procedimento relacionado ao clique no campo de nome.

Resultado da Aplicação da Modularização

Entrada:

O usuário preenche as informações no formulário:

- código;
- nome;
- descrição;
- tema;
- status.

Processamento:

O sistema organiza as funções em partes separadas:

- conexão com o banco de dados;
- validação das informações;
- criação da tabela;
- inserção dos dados;
- limpeza dos campos;
- controle dos eventos dos botões.

Cada procedimento executa uma função específica dentro do sistema.

Saída:

O sistema realiza:

- conexão com o banco de dados;
- criação automática da tabela;
- validação dos dados informados;
- inserção das informações no banco;
- exibição de mensagens de sucesso ou erro para o usuário.

Observações Reflexivas do Grupo (Modularização)

O grupo percebeu que a modularização foi muito importante para o desenvolvimento do projeto, pois ajudou a manter o código mais organizado e facilitou o trabalho em equipe. A separação das funcionalidades em procedimentos diferentes tornou o sistema mais fácil de entender e ajudou na identificação de erros durante o desenvolvimento. Além disso, ficou muito mais simples realizar alterações sem comprometer outras partes do programa. A experiência com essas estruturas mostrou que a modularização é essencial para desenvolver sistemas mais organizados e facilitar a manutenção da aplicação de forma plena.

Aplicação das Estruturas de Vetores e Matrizes no Projeto

Além da modularização, o projeto também utilizou estruturas de vetores e matrizes para ajudar na organização dos dados dentro do sistema.

O vetor foi utilizado para armazenar os status válidos dos jogos cadastrados. Nele foram definidos os valores "A" e "I", representando os status permitidos pelo sistema. Essa estrutura foi utilizada no procedimento `ValidarStatus()`, responsável por verificar se o usuário digitou um valor válido antes de inserir as informações no banco de dados.

```
private string[] statusValidos = { "A", "I" };
```

Com isso, o vetor ajudou a evitar erros de preenchimento e garantiu uma validação mais organizada das informações digitadas pelo usuário.

Exemplo da aplicação do vetor	
Vetor	Função
statusValidos = { "A", "I" }	Armazena os status válidos permitidos no sistema.

SA matriz foi utilizada para armazenar exemplos de jogos contendo código e nome. Os dados foram organizados em linhas e colunas, facilitando a visualização e organização das informações dentro do sistema.

Essa estrutura foi utilizada no procedimento `MostrarJogosExemplo()`, responsável por percorrer os dados armazenados e exibir as informações na tela para o usuário.

```
private string[,] jogosExemplo =  
{ "1", "Jogo1" },  
{ "2", "Jogo2" },  
{ "3", "Jogo3" }
```

Exemplo da aplicação da matriz	
Código	Nome do jogo
1	Jogo 1
2	Jogo 2
3	Jogo 3

Resultado da Aplicação dos Vetores e Matrizes

Entrada:

O usuário informa:

- código;
- nome;
- descrição;
- tema;
- status do jogo.

Processamento:

O sistema:

- utiliza o vetor para validar os status permitidos;
- utiliza a matriz para armazenar exemplos de jogos;
- verifica os dados antes de inserir as informações no banco.

Saída:

O sistema:

- permite apenas status válidos;
- exibe os jogos armazenados na matriz;
- realiza o cadastro das informações corretamente.

Observações Reflexivas do Grupo (Vetores e Matrizes)

O grupo percebeu que o uso de vetores e matrizes ajudou na organização das informações dentro do sistema. O vetor facilitou a validação dos dados digitados pelo usuário, enquanto a matriz ajudou a organizar e exibir informações de forma mais estruturada. A utilização dessas estruturas contribuiu para um melhor entendimento sobre organização de dados e lógica de programação durante o desenvolvimento do projeto.